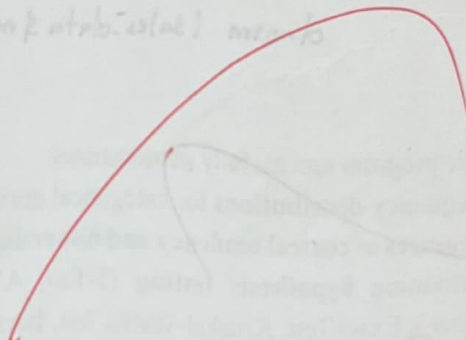


> print (crop.data)

	Temperature	Humidity	Rainfall	Soil moisture	Disease Risk
1				high	high
2	25	80	120	high	low
3	30	60	70	medium	low
	28	75	70	high	
4				low	high
	35	55	30		high
5		85	150	high	
6	22			high	low
	27	78	100		high
7		50		low	
8	33		20	medium	low
	29	70	85	medium	high
9	31	65	60		
10		88		high	low
	24		140		
11	34	52	25	low	
12		82	110	high	high



Ex:No: 4A

Date: 8/04/26

Predicting Crop Disease Risk Using Decision Tree Algorithm

Problem Description:

An agricultural research institute monitors environmental conditions to reduce crop losses caused by plant diseases. Factors such as temperature, humidity, rainfall, and soil moisture strongly influence disease occurrence. The institute wants to **predict whether crops are at high or low disease risk** based on these conditions using a **Decision Tree model**, enabling early preventive measures.

Objective:

To build and evaluate a Decision Tree classification model in R to predict crop disease risk using environmental data with train-test split and accuracy calculation.

Procedure:

- Step 1: Create the Crop Environment Dataset - crop_data

	Temperature	Humidity	Rainfall	SoilMoisture	DiseaseRisk
1	25	80	120	High	High
2	30	60	40	Medium	Low
3	28	75	90	High	High
4	35	55	30	Low	Low
5	22	85	150	High	High
6	27	78	100	High	High
7	33	50	20	Low	Low
8	29	70	85	Medium	High
9	31	65	60	Medium	Low
10	24	88	140	High	High
11	34	52	25	Low	Low
12	26	82	110	High	High

```
crop_data <- data.frame ( Temperature = c( 25, 30, 28, 35, 22, 27, 33, 29, 31,
24, 34, 26 ), Humidity = c( 80, 60, 75, 55, 85, 78, 50, 70, 65, 88, 52, 82 ),
Rainfall = c( 120, 40, 90, 30, 150, 100, 20, 85, 60, 140, 140, 110 ), Soilmoisture = c(
'High', 'medium', 'high', 'low', 'high', 'high', 'low', 'medium', 'medium', 'high', 'low',
'high' ), Diseases Risk = c( 'high', 'low', 'high', 'high', 'low', 'low', 'high',
'low', 'low', 'high', 'low', 'high' ) )
```

```
#Print crop_data
```

```
print (crop_data)
```

> str (exap_data)

'data-frame': 10 obs of 5 variables:

temperature: num 25 30 28 35 22 29 33 27 31 24
humidity: num 80 65 75 55 85 72 50 70 65 88
rainfall: num 120 90 70 30 50 100 20 85 60 140
soilmoisture: factor w/3 levels "High", "Low", "Medium": 1 3 1 2 1 1 1 1 1 1
diseaserisk: factor w/3 levels "High", "Low": 1, 2 1 2 1 1 1 1 1 1

> cat ("Training rows: ", nrow (train_data), "\n")

Training rows: 8

> cat ("Testing rows: ", nrow (test_data), "\n")

Testing rows: 4

> print (disease_tree)

n=8

node, split, n, loss, yval (y206)

* denotes terminal node

- 1) root & 2 high (0.750000 0.250000)
- 2) temperature < 30 6 0 High (1.0000 0.00000) *
- 3) temperature >= 30 2 0 Low (0.000000 1.00000) *

- Step 2: Convert Categorical Variables to Factors as.factor

```
crop_data$soilmoisture <- as.factor(crop_data$soilmoisture)
```

```
crop_data$DiseaseRisk <- as.factor(crop_data$DiseaseRisk)
```

```
# str() - crop_data
```

```
str(crop_data)
```

- Step 3: Train-Test Split - 70% training and 30% Testing

```
set.seed(123)
```

```
n <- nrow(crop_data)
```

```
train_index <- sample(1:n, size = 0.7 * n)
```

```
train_data <- crop_data[train_index,]; test_data <- crop_data[-train_index,]
```

```
# Print number of rows in train and test data
```

```
cat("Training rows:", nrow(train_data), "\n")
```

```
cat("Testing rows:", nrow(test_data), "\n")
```

- Step 4: Build the Decision Tree Model - disease_tree

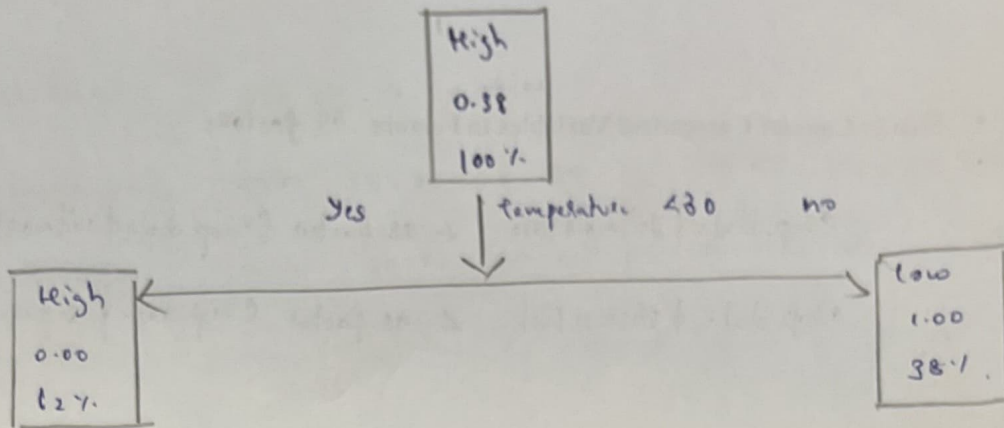
```
install.packages("xpart")
```

```
library(xpart)
```

```
disease_tree <- xpart(DiseaseRisk ~ Temperature + Humidity + Rainfall +  
Soilmoisture, data = train_data, method = "class")
```

```
print(disease_tree)
```

> rpart.plot (diagram tree)



> predicted_risk

2 4 5 7
low low high low
levels: high low

> confusion-matrix

	Predicted	
	High	Low
Actual High	1	0
Actual Low	0	3

> accuracy

0.75

```
#Print disease_tree
```

- Step 5: Visualize the Tree

```
install.packages("rpart.plot")  
library(rpart.plot)  
rpart.plot(disease_risk)
```

- Step 6: Predict Disease Risk

```
predictions <- predict(disease_tree, test_data, type = "class")  
  
print(predictions)
```

- Step 7: Confusion Matrix

```
conf_mat <- table(predicted = predictions, Actual = test_data$disease)  
  
print(conf_mat)
```

- Step 8: Accuracy Calculation

```
accuracy <- sum(diag(conf_matrix)) / sum(conf_mat)  
  
cat("Accuracy:", accuracy)
```

Result:

Thus, the program successfully demonstrates the use of the **Decision Tree algorithm** in R to predict crop disease risk based on environmental conditions. The model effectively classifies disease risk using a **train-test split and accuracy evaluation**, providing valuable insights for early agricultural intervention.

Q3 "first few rows of dataset:"

	sepal.length	sepal.width	petal.length	petal.width	species
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
3	4.7	3.2	1.3	0.2	Setosa
4	4.6	3.1	1.5	0.2	Setosa
5	5.0	3.6	1.4	0.2	Setosa
6	5.4	3.9	1.7	0.4	Setosa

Ex: No: 4B

Date: 08/04/24

Decision Tree Algorithm Using Iris Dataset

Problem Description:

The Iris dataset contains measurements of iris flowers belonging to three species: *Setosa*, *Versicolor*, and *Virginica*. The goal is to use flower measurements such as sepal length, sepal width, petal length, and petal width to **classify the species of iris flowers** using a **Decision Tree algorithm**.

Objective:

To implement a Decision Tree classification model in R using the Iris dataset, perform train-test splitting, predict flower species, and evaluate model accuracy.

Procedure:

→ Step 1: Install and Load Necessary Libraries

- Install and load rpart (for decision tree modeling) and rpart.plot (for visualization).
- Load caret for dataset splitting and evaluation.

```
install.packages ("rpart")
```

```
install.packages ("rpart.plot")
```

```
install.packages ("caret")
```

```
library (rpart)
```

• Step 2: Load and Explore the Dataset

- Use the built-in iris dataset.
- Print first six rows of iris dataset.

```
data (iris)
```

```
head (iris)
```


> print (tree_model)

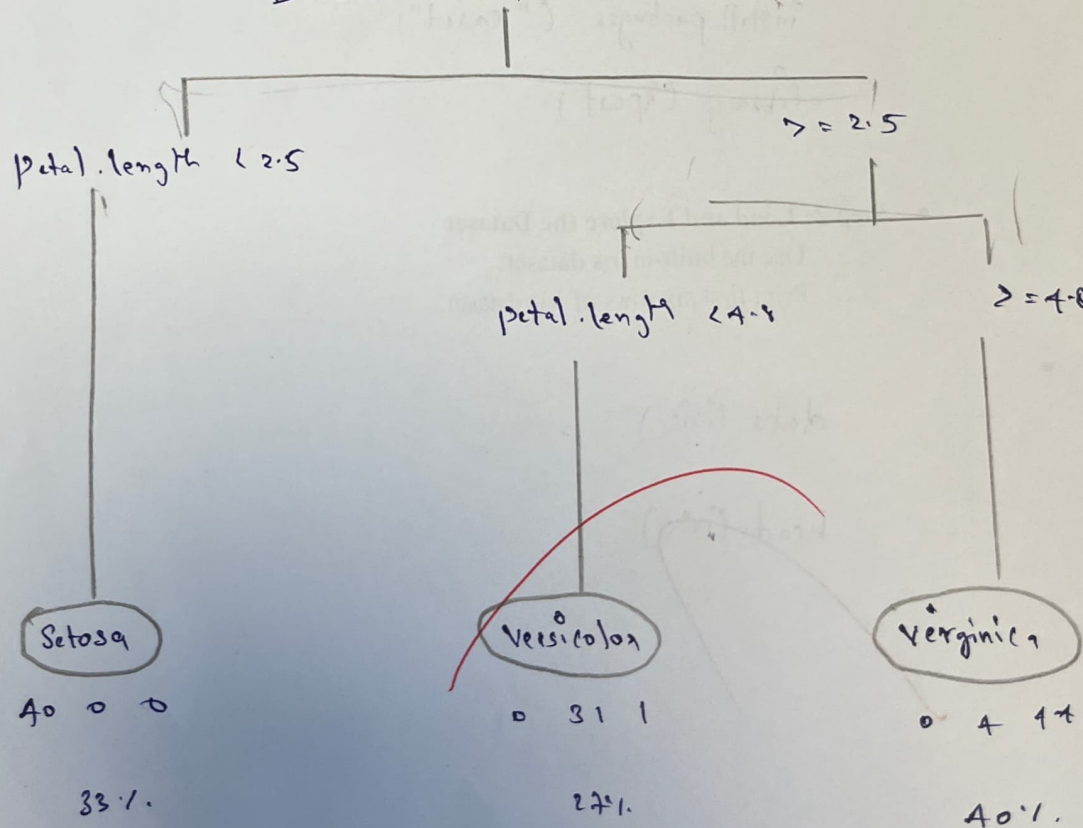
n=120

node 3, split 7, loss, yval, (yprob)

* denotes terminal node

- 1) root 120 75 Virginica (0.333333 0.2916667 0.3550000)
- 2) petal.length < 2.45 40 0 Setosa (1.000000 0.000000 0.000000)*
- 3) petal.length >= 2.45 80 35 Virginica (0.000000 0.4375000 0.5625000)
- 6) petal.length < 4.75 31 1 Versicolox (0.000000 0.9687500 0.0312500)*
- 7) petal.length >= 4.75 49 4 Virginica (0.000000 0.083333 0.9166667)*

Decision Tree for Iris Dataset



- **Step 3: Split the Dataset**

- 80% for training, 20% for testing using createDataPartition().

```
library(caret)
set.seed(123)
train_index <- createDataPartition(x = iris$Species, p = 0.8, list = FALSE)
train_data <- iris[train_index, ]
test_data <- iris[-train_index, ]
cat(nrow(train_data))
cat(nrow(test_data))
```

- **Step 4: Train the Decision Tree Model**

- Use the rpart() function to build the decision tree and print it.

```
library(rpart)
iris_tree <- rpart(Species ~ ., data = train_data, method = "class")
print(iris_tree)
```

- **Step 5: Visualize the Decision Tree**

- Use rpart.plot() to plot the decision tree structure.

```
library(rpart.plot)
rpart.plot(iris_tree)
```

Confusion matrix:

Confusion Matrix and Statistics

Referee

Prediction	setosa	Versicolour	Virginica
Setosa	10	0	0
Versicolour	0	13	0
Virginica	0	2	5

Overall statistics

Accuracy : 0.9233

95% CI : (0.7793, 0.9918)

No Information Rate : 0.5

P-Value [Acc > NIR] : 4.34E-02

Kappa : 0.8947

Test P-Value : NA

Statistics by class:

	class . setosa	class . versicolour	class . virginica
Sensitivity	1.00000	0.8689	1.00000
Specificity	1.00000	1.00000	0.9200
Pos pred value	1.00000	1.00000	0.7143
neg pred value	1.00000	0.8824	1.00000
prevalence	0.3333	0.5000	0.1667

Accuracy : 0.9333333

• Step 6: Evaluate the Model

- Predict on the test dataset.
- Compute the **confusion matrix** and accuracy score.

Make Predictions on Test Data

```
predictions <- predict (iris.tree, test.data, type = "class")
```

```
head(predictions)
```

Evaluate Model Performance Confusion Matrix and Accuracy

```
conf.mat <- confusionMatrix(predictions, test.data$Species)
```

```
print(conf.mat)
```

```
accuracy <- sum(diag(conf.mat)) / sum(conf.mat)
```

```
cat ("accuracy :", accuracy )
```

```
cat ("accuracy :", conf.mat$overall["Accuracy"],)
```

Result:

The Decision Tree model was successfully implemented and visualized in R using the iris dataset. The model achieved an accuracy of 93.33%, showing high classification performance. The decision tree structure was visualized, helping in understanding how the algorithm makes predictions based on feature splits.